

# 홍성문Portfolio

창의적인 문제해결 방법과, 몰입으로 완성하는 개발자 홍성문입니다.

## Contact

Hong sung-moon

010-2401-3487

sunmoonkr@gmail.com

# 이력 사항

창의적 문제해결 방법과, 몰입으로 성과를 만드는 개발자 홍성문입니다.



홍성문

SW 개발자

—

1999.05.27

010-2401-3487

sunmoonkr@gmail.com

- 2016.03~2018.02 홍익대학교 부속 고등학교 졸업
- 2020.03~2026.08 한양대학교 컴퓨터소프트웨어학부
- 2025.11~2026.01 ㉠화이트큐브 백엔드 개발 인턴
- 2025.03~재직중 ㉠코드박스 Software Engineer

## 주요 활동

- 2026.02~2026.03 BeatBuddy 서비스 개발 서포트
- 2025.07~2025.09 '언제볼래' 캘린더 앱 프로젝트
- 2025.04~2025.05 스윙프 웹 9기 활동. Fiteam 프로젝트
- 2025.02~2025.10 MODPlus 프로젝트
- 2025.01~2025.02 'TimeSpace' 웹 캘린더 프로젝트
- 2024.09~2024.12 Forif 개발 동아리 Web Frontend 활동
- 2023.12~2024.01 KAIST 몰입캠프 14기
- 2023.01~2023.02 LG Aimers 2기 수료
- 2020.03~2023.07 한양대학교 게임개발 동아리 오파츠 부회장
- 2020.03~2023.07 한양대학교 AI 학회 HAI

## 기타 활동

- 백준 골드2
- 프로그래머스 PCCP(코딩전문역량인증 시험) C++ Lv3

Hong sung-moon Portfolio

# Contents

[https://www.notion.so/1f0038bdc42b805f9b38c2d6aa0bda0b?source=copy\\_link](https://www.notion.so/1f0038bdc42b805f9b38c2d6aa0bda0b?source=copy_link)

1

## AI 검색 – 투자자의 IR탐색

코드박스 - 투자자가 원하는 IR을 빠르게 찾도록 도와주는 AI 검색기능

---

2

## BeatBuddy 프로젝트

공연, 클럽, 파티, 소규모 이벤트 참여를 위한 플랫폼 서비스

---

3

## 크롤러 시스템

화이트 큐브 인턴

---

4

## MODPlus 프로젝트

단백질의 PTM분석을 위한 Java 기반 소프트웨어 코드 리팩토링을 통한 성능개선

---

# AI 검색 기능

코드박스에서 Discovery Engine을 활용한 벡터 임베딩 기반의 AI검색 정확도 및 속도 개선

Details : [https://www.notion.so/AI-33b038bdc42b803f9cd1fabd56a75a9d?source=copy\\_link](https://www.notion.so/AI-33b038bdc42b803f9cd1fabd56a75a9d?source=copy_link)

## 설명

1. 기간 : 26.04.20 ~ 26.05.13 (약 3주)
2. 인원 : 개인(다른 Task와 병행작업)
3. 기술 : Django, 리액트, GCP, Gemini API

## 기존 AI 검색

LangGraph 에이전트가 자연어(검색어)를 이해하고 LLM API를 호출하여 LLM으로 사전 추출한 IR 데이터 요약본 (창업자, 팀, 산업 등)을 LLM에 넣고 검색

## 목표

1. 기존과 같이 자연어 기반 입력을 받는다.
2. 기존 AI 검색 속도를 대폭 개선해 5초 내외로 검색 결과를 사용자에게 보여준다.
3. 사용자의 입력에 대해 더 만족도 높은 결과를 제공한다.

## 평가지표

1. 기술관점 : 검색을 했을 때 K개 결과 중 실제로 관련 있는 IR의 비율. K개의 순서를 관련도 기준으로 나오는지.
2. 비즈니스 성과 지표 : 검색 했을 때의 상세보기 + '팔로우/커피챗' 클릭 수를 기준으로.

Q 그린펀드 투자 가능한 다테크 초기기업 찾아줘.



"그린펀드 투자 가능한 다테크 초기기업 찾아줘."에 맞는 IR을 찾았어요.



퓨리오

Seed · 2억 희망

환경 위성 기술 플랫폼 · 2025년 설립

HydriOx는 공기와 수환경의 오염 물질을 근본적으로 분해하는 플랫폼입니다.



그린루프

Pre-A · 15억 희망

IoT 의류 수거 시스템 판매, 수거 의류 유통 및 판매, 데이터 판매 · 2024년 설립

IoT 기반의 의류 자원 순환 시스템



이이키

Seed · 1억 희망

업사이클링 원료 플랫폼 · 2025년 설립

폐기되는 당근잎을 고부가가치 기능성 원료로 전환해 B2C 제품과 B2B 원료 사업을 동시에 전개하는 업사이클링 푸드 기업입니다.

# 개선된 검색 Flow

DE path+ DB-path 결과를 Merge 해서 응답, 입력후 결과 출력까지 대략 4~7초 정도(가벼운 LLM이 2개)

1

## LLM Parser

사용자의 입력 내용을 키워드 기반 두 가지 경로로 분리

- - 룰 매칭 부분 (예: "팁스 희망", "시리즈 A", "5억 이상", "서울대 출신") → DB 조건 - 나머지 자연어 (예: "EV 충전") → DE 의미 검색
- - 검색 입력 예시: "그린펀드로 투자할 수 있는 기후테크 딜 보여줘. 3년 이상된 기업으로 보여줘"

```
{  
  "matched_rules": [  
    {"rule_id": 100, "value": null},  
    {"rule_id": 40, "value": "3년 이상된"  
  ],  
  "residual_phrases": ["기후테크 딜"],  
  "trash": ["로", "투자할", "수", "있는",  
    "기업", "으로", "보여줘", "."]  
}
```

2

## Discovery Engine Path

PDF 본문 의미 검색 + LLM FP filter 로 노이즈 제거

- - Synonyms 자동 확장 → "기후테크 딜 그린펀드 녹색기후기금 친환경"
- - 확장된 query 로 IR PDF 본문 chunk 의미 매칭 (Discovery Engine v1 + v3)
- - AnswerQuery LLM이 의미적으로 잘못 잡힌 chunk reject
- - rel, sem, keyword 점수 계산 기준으로 유사도 판단

3

## DB Path

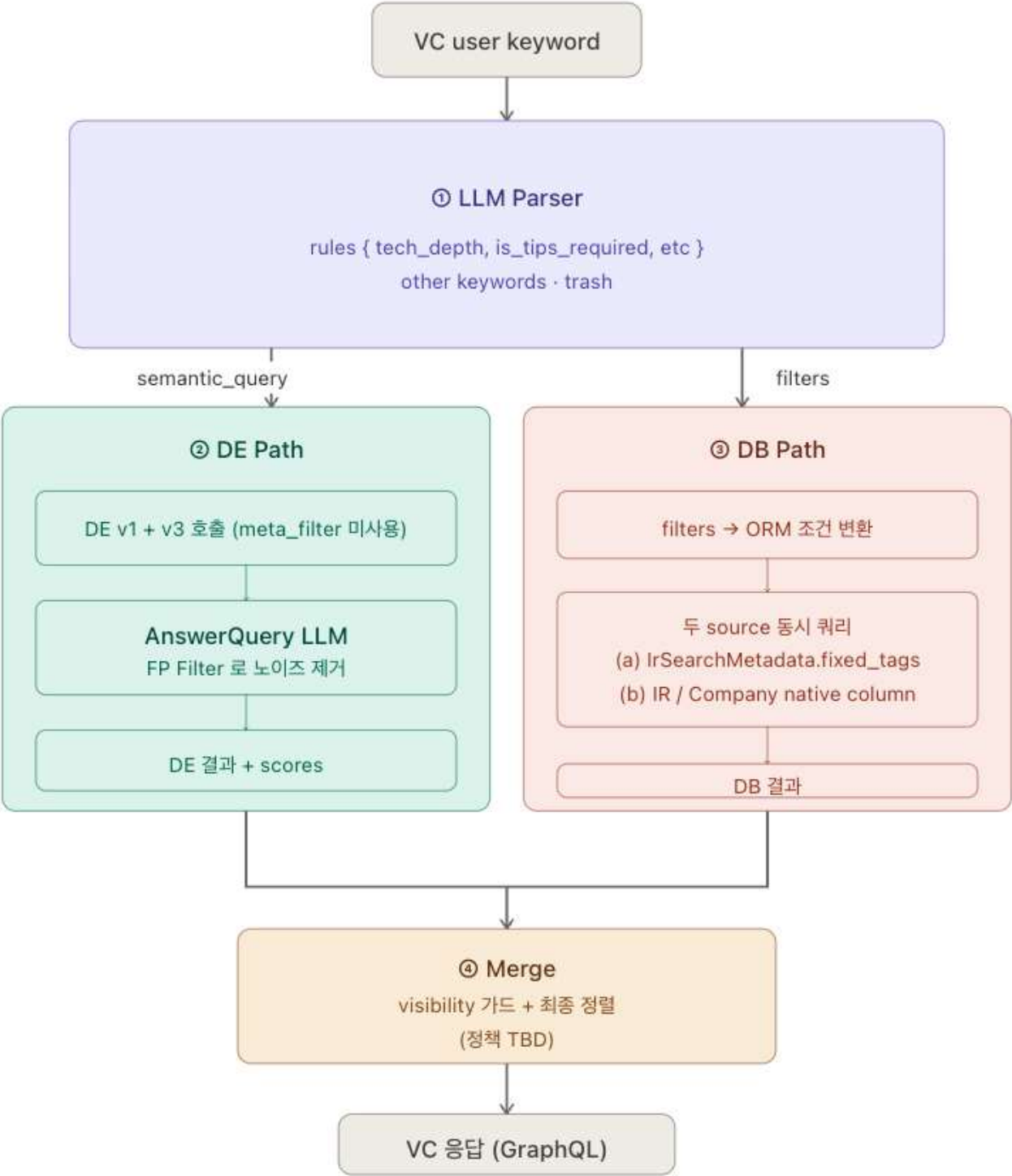
IR/Company native 컬럼 + LLM 메타 태그 Json 데이터 동시 쿼리

- - matched\_rules (rule\_id 100, rule\_id 40) → DbFiltersBuilder 가 ORM 쿼리로 변환
- - parametric 룰의 value ("3년 이상된") 는 Python phrase parser 가 결정적 해석 → FoundingAgeRule(min\_years=3)
- - 룰을 추가하고 싶다 -> registry.py 에다가 고유id 부여해서 추가가능



# Flow 아키텍처 & 핵심 기술 결정과 효과

Read side — 키워드 → SearchPlan → 이중 경로 → Merge



## (1) LLM Parser — 자연어 키워드 3 갈래 분리

- Gemini 2.0 Flash 로 검색어를 3 영역으로 결정적 (temperature=0 + response\_schema 강제) 분리:

```
matched_rules    - 사전 정의 DB 룰 매칭 (예: "팁스 희망" → is_tips_required=True)
residual_phrases - DE 의미 검색으로 보낼 자연어 (예: "EV 충전")
trash            - 검색에 무의미한 어휘 ("으로", "보여줘", "기업", "." 등)
                  DE query 에서 제외
```

- 왜 이렇게 했나 : 단일 LLM 매칭: 룰 substring + LLM 하이브리드 대신 LLM 단일. 룰 description 한 곳만 유지하면 동의어 / 약어 / 영문 변형 자동 흡수.
- parametric 룰의 환각 격리: "5년 이상" 같은 표현의 숫자 / 단위 변환은 LLM 에게 안 맡김. LLM 은 사용자 원문 phrase 발체만, Python 결정적 파서가 해석 → FoundingAgeRule(min\_years=5).
- trash 분리의 부수 효과: DE ranking 노이즈 ↓ + 후속 LLM (AnswerQuery) → 입력 크기 감소 → 미세한 latency 단축.

## (2) AnswerQuery 기반 LLM FP Filter — DE False Positive 제거

Vertex AI Discovery Engine 의 AnswerQuery API 로 DE 가 매칭한 결과를 LLM 이 한 번 더 검증.

- DE 의 토큰 매칭이 의미적 false positive 발생 — 예: "태양 에너지"
- 검색에 자동차 회사 IR 의 광고 이미지 텍스트가 매칭. 사람이 보면명백히 무관한 결과.

**속도의 핵심1** - 출력 포맷 : {"rejected": ["ir-1252", "ir-879"]} . JSON 으로 reject 할 IR ID list 만 강제. 출력 토큰이 사실상 무시 가능 수준 → 입력이 ~65K 토큰 이어도 latency 1-2초 수준 유지.

**속도의 핵심2** : 입력이 IR PDF 원본이 아니라 DE 가 매칭했다고 알려준 텍스트(매칭된 chunk ± 앞뒤 문맥) 만 inject. 실제 IR 자료 전체보다 훨씬 작은 컨텍스트.

**Model** : AnswerQuery 는 Google 의 grounded answer 모델 (Discovery Engine 의 내장 RAG 튜닝된 빠른 모델). Vertex AI 의 gemini-flash 직접 호출이 아님 → grounded 답변 특성상 환각 ↓, 응답 짧을 때 latency ↓.

# BeatBuddy 프로젝트

BeatBuddy는 공연, 클럽, 파티, 소규모 이벤트 참여를 위한 플랫폼 서비스입니다.

테트라포드 1기 활동으로 비트버디 팀에 BE로 합류해 이벤트 캘린더, 결제 시스템 등을 개발했습니다.

## 소개

기간 : 2026.01.31 ~ 2026.03.05

인원 : PM 1, PD 2, FE 2, BE 2

역할 : Backend Developer, Reviewer

주요 키워드 : Spring Boot, MySQL, Kafka, Redis/Caffeine, Toss Payments

1

## 내가 맡은 역할

- 백엔드 개발
- 캘린더 월 조회 API 및 ETag 기반 캐싱
- 알림 기능 및 Kafka 기반 비동기 처리
- 결제/환불 API 설계 및 후속 처리 구조 설계
- FE/BE 리뷰어 역할 수행
- AI-agent Guide로 생산성 향상

2

## BeatBuddy에 필요한 기능

- 참여 또는 관심있는 이벤트를 한눈에 확인하고, 메모와 사진 저장도 가능한 이벤트 캘린더
- 외국인 사용자 편의성을 높이기 위한 다국어 번역 기능
- 국내·해외 사용자를 모두 고려한 간편결제 기능
- 술자리에서 즐기는 미니게임 제작

3





# 결제 기능 설계

서버 승인 기반 비동기 결제 플로우.

Details : [https://www.notion.so/BeatBuddy-302038bdc42b806a87a7d545d4521e12?source=copy\\_link](https://www.notion.so/BeatBuddy-302038bdc42b806a87a7d545d4521e12?source=copy_link)

## 비즈니스 요구사항과 서비스 특성

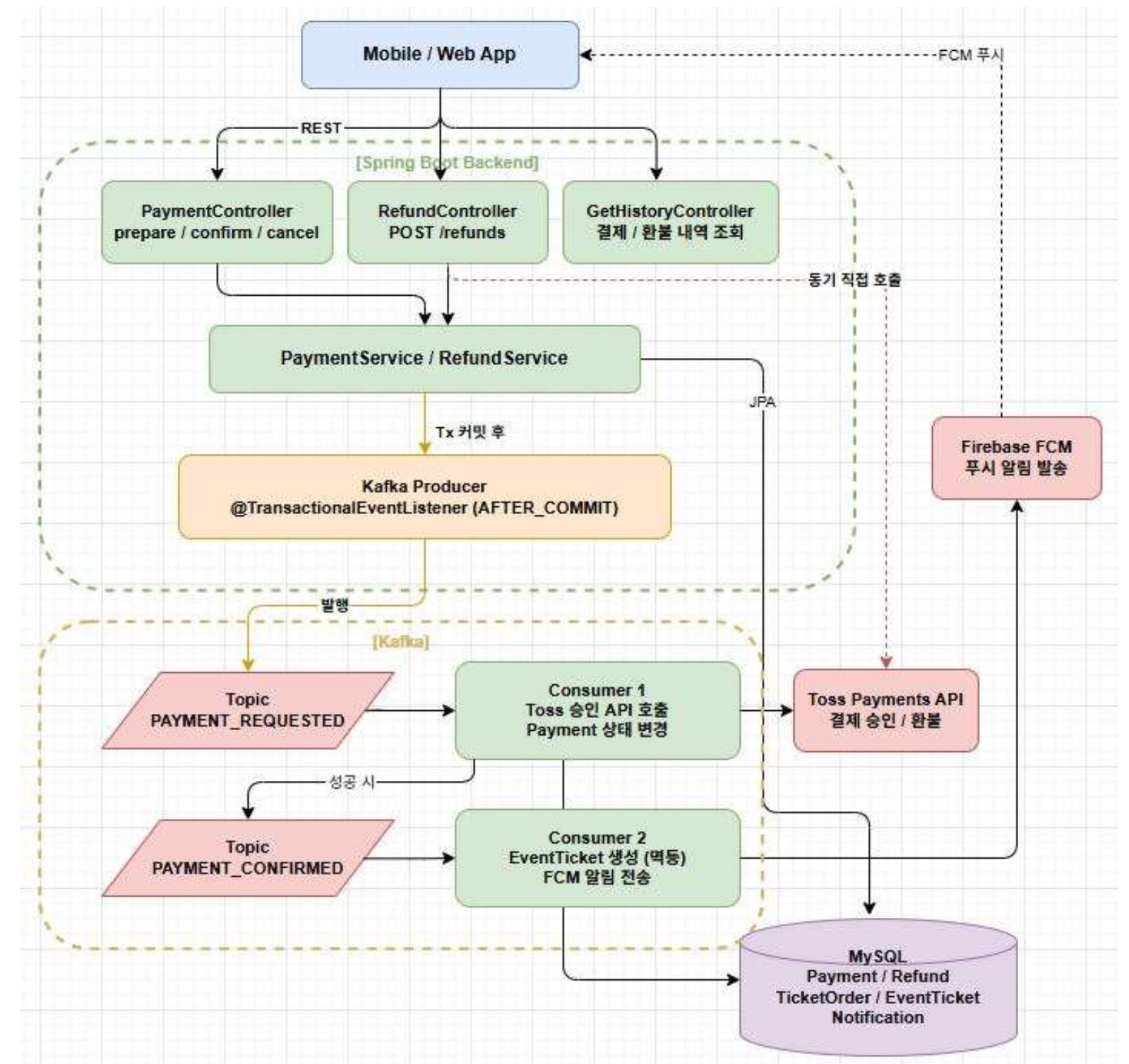
- 기존에는 계좌이체 후 수동으로 관리하는 방식이었다.
- 결제 이후에도 **티켓 발급, 참여자 반영, 알림 전송, 환불, 결제 내역 조회**까지 하나의 서비스 흐름으로 이어져야 한다.
- 대형 티켓팅 수준의 폭발적 개발 관점에서의 해석과 설계 트래픽은 아니므로, **현재 규모에 과하지 않으면서도 이후 확장 가능한 구조**가 필요하다.

4

## 정합성과 운영 안정성을 갖춘 결제 구조 설계

- **Toss Payments + PayPal** 두가지로 국내/해외 결제를 함께 구현
- 프론트 결과가 아니라 **서버 승인 기준**으로 결제 완료 여부를 판단
- 결제 이후 티켓 발급·참여자 반영·알림 전송은 **Kafka 기반 비동기 처리**로 분리
- 주문, 결제 시도, 이벤트 로그를 남겨 **중복 요청 방지**와 **상태 추적**이 가능하도록 구성
- 오류없는 결제 시스템을 위해 **AFTER\_COMMIT + Outbox/INbox 패턴 + Kafka + 멍등성 검증 + 이벤트 로그** 조합으로 안정성 확보

5





# 고민과 선택

## 1. 왜 Kafka를 선택해 비동기 처리로 분리했는가

결제 성공 이후에는 티켓 발급, 참여자 반영, 알림 전송까지 이어져야 했다. 이를 하나의 요청에서 모두 동기 처리하면 응답 시간이 길어지고, 일부 후속 작업 실패가 전체 결제 흐름에 영향을 줄 수 있었다. 그래서 결제 승인 이후 작업은 Kafka 기반 비동기 처리로 분리해 응답 책임을 줄이고 장애 전파 범위를 낮췄다.

1

## 3. 왜 결제 완료 판단을 서버 승인 기준으로 두었는가

클라이언트가 successUrl로 돌아왔다고 해서 결제가 최종 완료된 것은 아니다. 실제 결제 완료 여부는 서버가 Toss 승인 API를 호출해 검증한 뒤 확정해야 한다. 그래서 FE에서 결제 요청을 받으면, 결제 준비 API를 먼저 호출한 다음, FE에서 간편결제를 진행, 완료 되면 서버가 주문 정보와 결제 정보를 최종 확인한 뒤 최종 승인하는 구조로 설계했다.

3

## 2. 초기에 Outbox 패턴을 도입하지 않은 이유

Outbox 패턴은 좋은 선택이었지만, BeatBuddy의 당시 트래픽 규모와 개발 가능 시간을 고려하면 운영 복잡도와 비용이 더 컸다. 초대형 티켓팅 서비스처럼 동시 폭주가 발생하는 구조는 아니었기 때문에, 현재 단계에서는 AFTER\_COMMIT 이벤트 발행, Kafka 비동기 처리, 멍등성 검증, 이벤트 로그 기록 조합으로 현실적인 안정성을 확보하는 쪽이 시간상 더 적절하다고 판단했다. 이후 5월에 주말 시간을 활용해 OUTBOX/INbox 패턴을 도입했다.

2

## 4. 캘린더 월 조회 API와 ETag 기반 캐싱

캘린더 월 조회는 같은 달을 반복해서 확인하는 경우가 많아, 매번 DB를 조회하는 구조가 비효율적이었다. 그래서 Redis에는 월별 **ETag 버전만 저장**하고, 변경이 없으면 **304**를 반환하는 방식으로 설계했다.

Redis에는 월별 버전만 저장하고, If-None-Match 비교 후 변경 없으면 304 응답 데이터 자체를 캐싱하지 않아 메모리 부담이 적고, DB 조회와 응답 바디 전송을 함께 줄일 수 있음

4

# Crawler System 프로젝트

화이트큐브 백엔드 개발자 인턴으로 수행한 크롤링 시스템 설계 및 운영 프로젝트 경험

Details : [https://www.notion.so/Crawler-2e4038bdc42b80f5a37dfb33987440f2?source=copy\\_link](https://www.notion.so/Crawler-2e4038bdc42b80f5a37dfb33987440f2?source=copy_link)

## Situation

- 회사 내 크롤링 요구가 커지면서 팀별로 크롤러가 따로 생겼고, 언어/실행 방식/배포 환경이 제각각이었다.
- 작업을 안전하게 분산 실행하고, 결과를 신뢰성 있게 전달하며, 새로 추가되는 크롤링 기능 추가 요청들을 개발하는 확장성 있는 인프라가 필요했다.
- 특히 모든 작업이 단건 요청으로 처리되었고, 이때문에 시간적 비효율성 문제가 매우 컸다.

1

## Action

- 크롤링은 작업 시간이 길고 변동 폭이 커서 SQS 를 도입해 크롤링 작업을 관리.
- ECS Fargate, Local Computer 기반 Crawler Agent 구현. 데이터 크롤링 작업을 수행하는 워커와 CallBack을 담당하는 워커를 분리하고 Redis로 연결.
- Crawler API + Scheduler(Control Plane) 를 통해 Job 생성/검증/상태추적을 담당.
- SQS에서 작업을 받아 Crawler Agent 가 크롤링 기능을 담당.

3

## Task

- 크롤링 요청을 Job 단위로 표준화하고, 공통 파이프라인을 제공한다.
- 작업 규모가 커져도 처리 지연이 폭증하지 않도록 수평 확장 가능한 실행 구조를 만든다.
- "중복 처리 / 실패 재시도 / 배포 중 종료" 같은 운영 이슈에서도 결과 정합성을 유지한다.
- 작업이 실제로 어떻게 돌아가는지(큐 적체, 워커 상태, 성공/실패)를 운영자가 즉시 볼 수 있게 만든다.

2

## Result

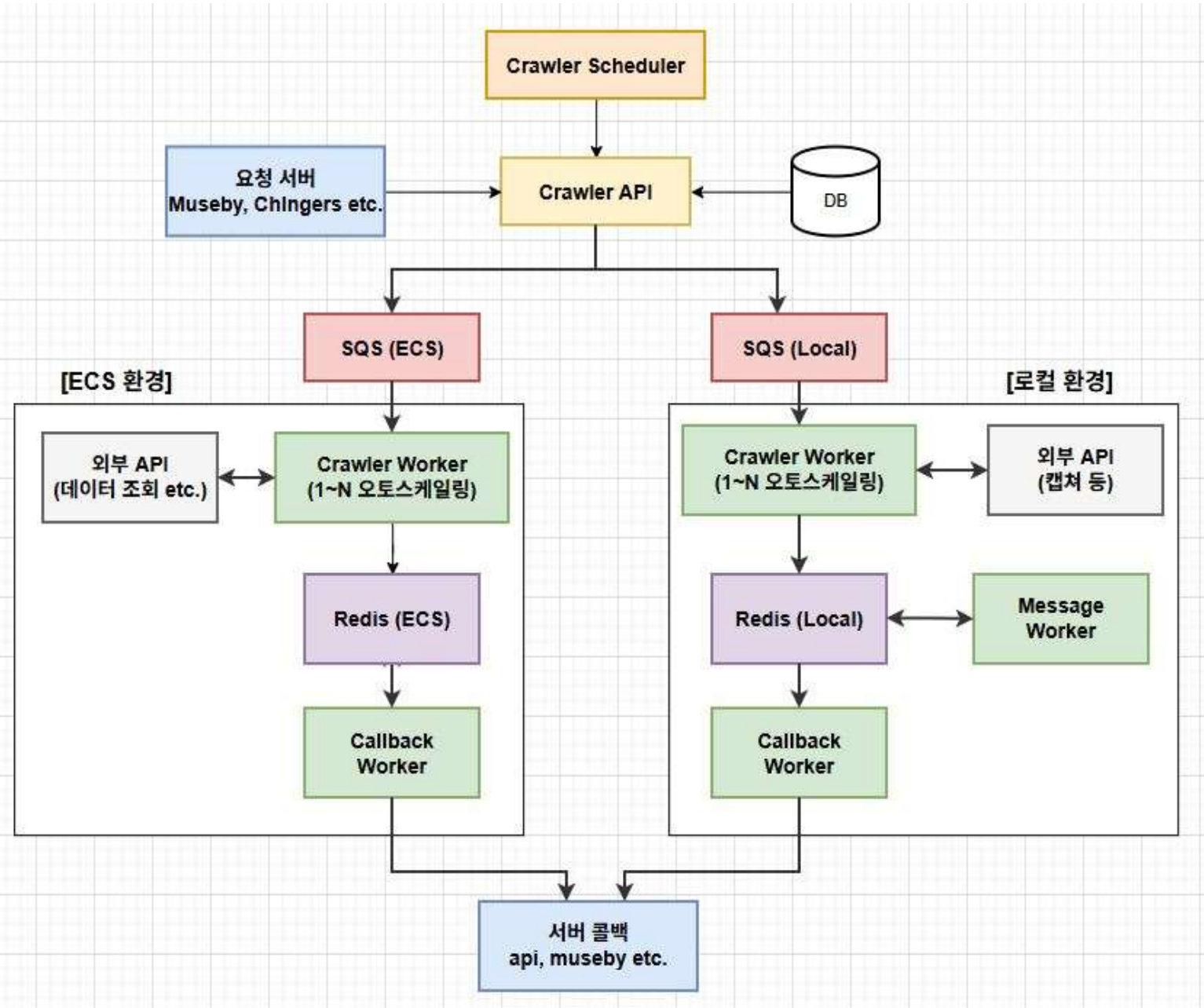
- 주기적 데이터 업데이트 같은 대량 요청을 Job에서 작은 Task 단위로 분해하고 결과를 집계하는 방식을 통해 **콜백 호출을 100 → 1로 축소**했다.
- 실패는 전체 재시도가 아니라 **실패 task만 재시도**하도록 설계해 비용과 지연을 줄였다.
- 멍등성 + graceful shutdown + DLQ 격리로 **중복 처리/무한 재시도/배포 중 작업 유실** 리스크를 통제.
- 로그인을 세션ID 저장 및 Refresh 방식으로 만들어 크롤링 작업 시간 단축

4

# Crawler System Architecture

비즈니스적인 니즈를 위해 데이터 수집과 자동화가 중요했고, 이를 안정적으로 수행하기 위한 크롤링 시스템 개발

## 아키텍처



1. **crawler-scheduler**: (Kubernetes) 특정 시간/정책에 따라 크롤링 Job을 생성해 **SQS에 enqueueing**.

2. **crawler-api**: 크롤링 Job 요청/관리/하트비트/모니터링

3. **crawler-agent**: AWS ECS, Local Computer 환경에서 실행한 워커가 SQS 메시지를 Pooling하여 크롤링 수행 → 중간 상태 저장(Redis)→ CallBack으로 결과 전달

### 워커 분리 설계 (Crawler / Callback / Message)

1. **작업 성격이 다름**: 크롤링은 "무겁고 오래 걸림", 콜백은 "짧고 자주 호출됨"
2. **병목 지점이 다름**: 크롤링은 CPU/네트워크/페이지 로딩, 콜백은 외부 API 호출/RateLimit
3. **확장 전략이 다름**: 크롤링만 스케일하면 되는데 콜백까지 같이 스케일하면 비용만 증가, Message 워커는 특정 작업에서만 사용됨.

**Redis 사용** : 큐 기반 분산 환경에서 필요한 중간 결과 저장 + 집계 + 멍등성 + 락/상태 관리를 빠르게 처리할 수 있기 때문에

1. **중간 결과 저장**: N개의 Task 결과를 누적해 "집계 콜백"이 가능하게 함
2. **완료 카운터/집계 트리거**: 작업 완료 수를 카운트해서 "전체 완료 시 콜백" 흐름 구성
3. **멍등성 키**: SQS 재전달/중복 처리 상황에서 결과 중복 저장 방지
4. **콜백 락/상태 플래그**: 콜백 워커가 여러 개일 때도 "한 번만 콜백"되게 제어



# Crawler System Solution

프로젝트를 수행하며 마주한 Problem과 Solutions

## 1. 크롤링 기능의 성능개선

### Problem

1. 단건 크롤링마다 로그인(15~30초)이 반복되어, 3,000명 업데이트 같은 배치에서 처리 시간 문제.
2. 프로필 업데이트를 1개 Job에서 for-loop로 처리하며 매 건 콜백 저장하던 구조가 대량 요청에서 콜백/비용 증가

### Solution

1. session\_id를 Redis에 캐싱하고 주기적으로 리프레시해 로그인 반복을 제거. 크롤링 기능 요구사항이 증가하는 상황에서 로그인 로직을 인스타그램 공통 로직으로 분리.
- 2.요청을 task로 fan-out하고 Crawler Worker(수집) / Callback Worker(집계 콜백 1회) 로 분리함.
3. 결과는 **부분 성공 모델**로 관리하고, 재시도는 **실패 task만** 수행해 실패 전파와 재처리 비용을 줄임

1

## 2. 운영 안정성

### Problem

1. **중복 처리 발생 가능성** : sqs 기반 워커는 처리 시간이 길거나(크롤링) 워커가 비정상 종료/배포로 끊기면 메시지가 다시 전달될 수 있음.
2. 배포/스케일 인 시 작업 유실 또는 재처리 폭주
3. 실패 유형이 섞여 있어 재시도 정책이 설립 필요

### Solution

1. **멱등성(Idempotency) 적용으로 중복 처리 차단**, 작업 단위를 (job\_id + task\_index)로 고정하고 이를 멱등키로 사용. Callback도 중복 되지 않게 Key 활용.
2. **graceful shutdown**으로 배포/스케일 안정성 확보
3. 재시도 분류 : Retryable (네트워크 타임아웃, 일시적 5xx, 일시적 rate limit) vs Non-retryable (계정 제한/인증 실패/메시지 포맷 오류)

2

## 3. 모니터링, 헬스, 큐, 표준화(trace\_id)

### Problem

1. 장애가 나면 원인이 큐 적체 / 워커 문제 / DB-Redis-SQS 장애 / 외부 콜백 실패 중 어디인지 빠르게 구분이 어려움.
2. 요청 단위 추적이 안 되면(로그 연결 끊김) 재현/원인 분석 시간이 길어짐.

### Solution

1. Health: DB/Redis/SQS 상태 + SQS visible/in-flight 지표를 한 번에 제공해 병목을 즉시 분리.
2. 로그 표준화: 모든 주요 로그에 job\_id/trace\_id를 포함해 end-to-end 추적 가능하게 구성.
3. 운영 지표/알림: DLQ 유입률, 콜백 실패율/지연을 지표화하고, 임계치 초과 시 Slack 알림으로 운영 개입 포인트를 명확히 함.

3

# MODPlus 프로젝트

코드적인 설명은 Notion에 자세히 적었습니다.

[https://www.notion.so/MODPlus-1f1038bdc42b803aa2c7c1b25b63cc33?source=copy\\_link](https://www.notion.so/MODPlus-1f1038bdc42b803aa2c7c1b25b63cc33?source=copy_link)

## 프로젝트 목적

기존 MODPlus의 Unrestrictive Search를 **멀티스레드 기반으로 리팩토링**하여, 단일 스레드와 동등한 **탐색 정확도를 유지**하면서 수행 시간을 대폭 단축하고, CPU 코어 수 증가에 비례해 확장되는 실행 구조를 구현한다.

**정확도 보존**: 싱글스레드 기준과 결과 동등성(Deterministic Merge) 보장.

**성능 향상**: 멀티코어 환경에서 선형에 가까운 스케일링 달성(36코어).

**병목 제거**: 전역 가변 상태의 슬롯 인덱스화로 락/경합 제거 및 핫패스 I/O 차단.

**메모리/GC 최적화**: 사전 할당 + 재사용(reset) 으로 GC Pause 최소화.

1

## 성과

**N코어 병렬화** + Slot **아키텍처**로 핫패스 락 제거, 고정 스레드풀:1:1 바인딩 고정할당/리셋으로 GC·컨텍스트 스위칭 최소화.

정확도 100% 일치-> Python비교 코드를 만들어 전 데이터셋 검증.

안정성·품질 개선: dirty code 정리 알고리즘 개선, 프로파일러 기반 미세 최적화로 추가 +17% 성능 향상.

**성과** : 36코어 환경에서 기존보다 36배 이상 빠르게 분석되는 소프트웨어로 개선.

2

## 프로젝트 진행

졸업 프로젝트 진행 기간 : 2025-02-25 ~ 2025-10월 심사  
개인으로 진행한 프로젝트였고, 다른 참가자들 보다  
성능이 가장 좋습니다.

3

MODPlus | 17751/17752

MODPlus | 17752/17752

[MOD-Plus] Elapsed Time : 141 Sec

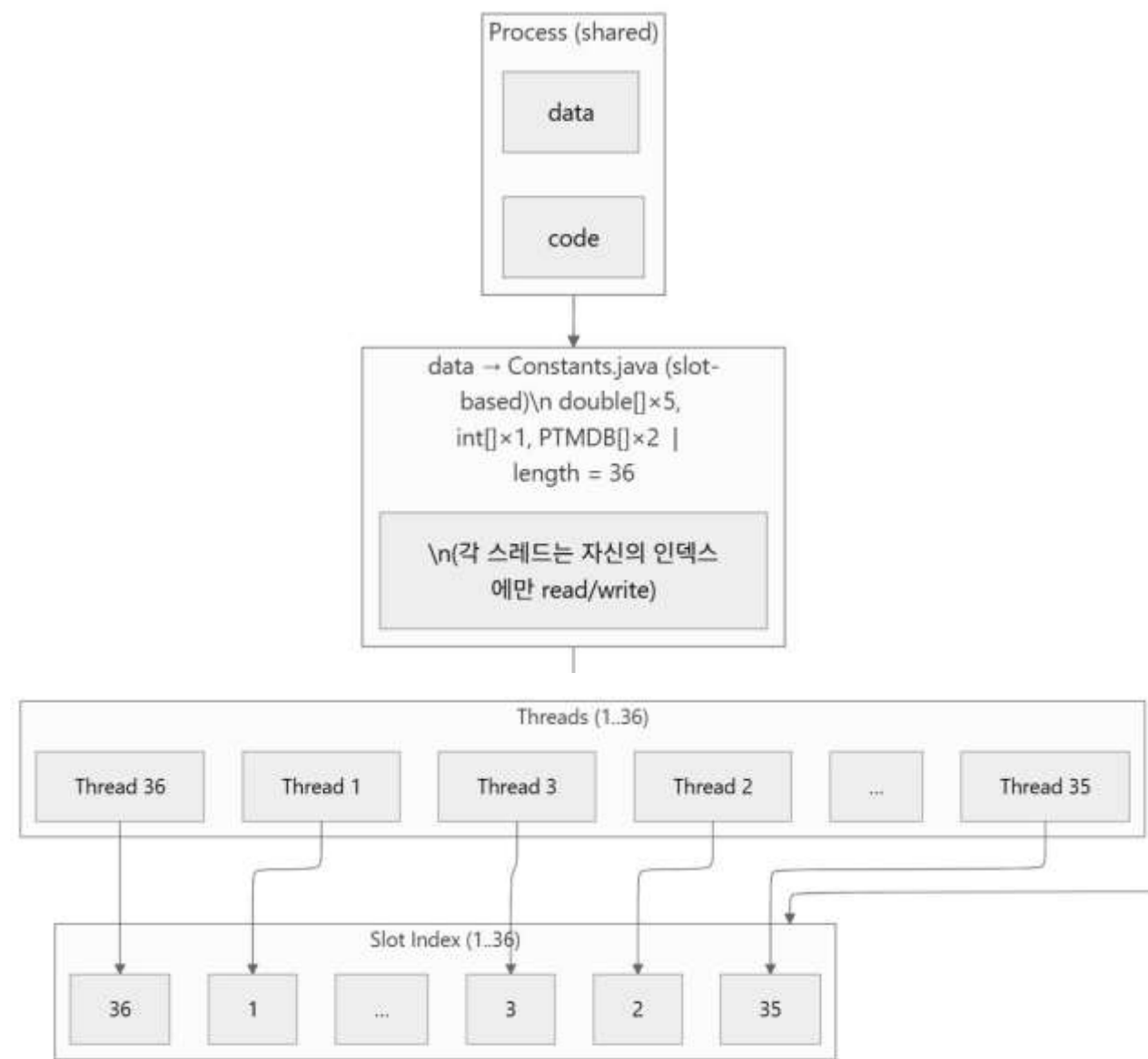
# MODPlus 프로젝트

성과를 만든 3가지 포인트

|   |                        |                                                                             |                                                           |
|---|------------------------|-----------------------------------------------------------------------------|-----------------------------------------------------------|
| 1 | Executor + Future      | 고정 스레드풀로 CPU-bound 작업<br>병렬화, 슬롯 바인딩 재사용                                    | 병렬화 0.8 효율.                                               |
| 2 | 알고리즘, 자료구조             | 1. StringBuffer -> String<br>2. 필드 로컬캐싱, 연산감소<br>3. LinkedList -> ArrayList | 17~18% 성능향상                                               |
| 3 | JVM-Garbage Collection | 기본 GC인 G1 -> Parallel GC 설정<br>Young 메모리 확장                                 | Parallel GC로 설정만 해도 5% 처리량 향상<br>메모리 특성까지 튜닝하면 5~10% 성능향상 |

# 1. 병렬화 구조

## 아키텍처



4

## 병렬처리 방법론



### Slot 형태의 배열(나의 선택)



### Thread Local 방식

기존 MODPlus는 'set parameter' → 펩타이드 1개 분석 → parameter 초기화 -> 펩타이드 1개 분석의 구조를 반복하는 방식이었으며, 분석 과정에 사용되는 다수의 전역 변수들이 병렬 처리에 적합하지 않은 구조로 구성되어 있었습니다. 이에 따라 스레드 안전성과 성능을 동시에 확보하기 위해, 분석 과정에서 변경되는 전역 변수와 객체 8개를 Slot 구조(Thread 갯수만큼 배열 생성)로 변환하고, 각 스레드에서 index 기반으로 접근하도록 설계했습니다. 함수 중에서도 동시성 문제가 되는 것들은 수정이 필요했습니다. ThreadLocal 방식에 비해, Slot 구조가 약 5% 더 빠른 성능을 보였고, 객체 생성 및 GC 부하를 줄여 메모리 사용량 또한 조금이지만 절감하는 효과가 있었습니다.

5



## 2. 알고리즘, 자료구조 개선

```
65 public String getPeptide(int s, int e){ | 13 usages
66     252 ms
67     2,260 ms
68     2,711 ms
69
70     816 ms
71 }

76 public String getPeptide(int s, int e) { 13 usages
77     960 ms
78     return new String(sequence, s, length: e - s, StandardCharsets.ISO_8859_1);
79 }
```

```
public CandidateContainer( LinkedList<TagPeptide> hmap, TagTrie trie ){ //multi-mod
    Collections.sort( hmap, new TagPeptComparator() );

public CandidateContainer(List<TagPeptide> hmap, TagTrie trie) { // multi-mod
    ArrayList<TagPeptide> list = new ArrayList<TagPeptide>(hmap);
    list.sort(new TagPeptComparator());
```

문자열 처리 방식을 수정해서 성능 개선  
StringBuffer 대신에 String을 사용해서 내부 재할당 문제도 해결하고, GC부담도 완화했다.  
getPeptide() 함수는 호출 빈도가 높은 함수이다.  
해당 최적화로 인해 **전체기준 약 3% 시간 단축**이 되었다.

6

sort할 데이터 갯수를 확인해 보니 적게는 7~8천개 에서 많게는 5만개 이상도 있었다.  
로직은 그대로 두고(sort 이후 부분은 getFirst/removeFirst 대신 인덱스 순회로 구현), 자료구조만 LinkedList에서 ArrayList 로 변경했다.  
**164초→155초** 로 무려 10초 가까이 성능이 개선되었다.

7

# 1. Garbage Collection

결과 테이블 - Parallel GC가 처리량(throughput) 최적

| 케이스 | GC            | 주요 튜닝                 | 총 시간          | GC 특성/메모                                            |
|-----|---------------|-----------------------|---------------|-----------------------------------------------------|
| A1  | G1 (기본)       | 기본값                   | 155 s         | Young 510회, GC 9.7 s                                |
| A2  | G1 (변형)       | Survivor ↑            | 165 s         | Young 2배↑, 22 s GC (Eden ↓ 부작용)                     |
| B1  | ZGC           | 기본값                   | 202 s         | 스레드 경합/배치형 워크로드에 부적합 추정                             |
| C1  | Parallel      | 기본값                   | 147 s         | Stop-the-world 총 26.4 s (G1보다 큼) 그러나 앱 실행 145→120 s |
| C2  | Parallel      | Young ≈ 6 GB (총 8 GB) | 141 s         | 최적 지점 근처, Young 7 GB는 오히려 GC시간↑                     |
| D   | 대형 입력(335 MB) | G1 vs Parallel        | 1056 → 1004 s | 약 5% 단축, 규모 커져도 일정한 비율로 시간 단축                       |

**Parallel GC가 유리한 이유** : Stop-the-world는 길지만 수집 자체를 다중 코어로 압축 처리 → 총 수집시간/총 실행시간 단축. 배치성 워크로드는 지연보다 처리량이 중요. 소프트웨어 특성상 Young 영역 특히 Eden 영역이 가장 많이 필요함.

# Thank you

홍성문

Contact

010-2401-3487

[sunmoonkr@gmail.com](mailto:sunmoonkr@gmail.com)